Chương 4 (Phần 1): Phát hiện lỗ hổng phần mềm – Phân tích động

4.1 Giới thiệu kiểm thử bảo mật – 4.2 Phủ mã nguồn – 4.3 Theo dõi thực thi chương trình

ThS. Vũ Quang Dũng

Khoa Hệ thống thông tin – Trường Đại học Phenikaa

CSE703093 – An toàn phần mềm

Nhắc lại toàn bộ hành trình học phần

- Chương 1: Khái niệm bảo mật, Kripke structure, testing/verification cơ bản
- Chương 2: Lỗi bộ nhớ C cụ thể, công cụ static analysis thực tế
- Chương 3: Nền tảng toán học (SAT/SMT), Bounded Model Checking, Memory Model
- Chương 4 chuyển hướng sang **phân tích động** – kiểm thử bằng cách thực sự *chạy* chương trình, kết hợp với các kỹ thuật hình thức đã học để tạo ra fuzzing thông minh

Nội dung Phần 1

1. 4.1 Giới thiệu về kiểm thử bảo mật
2. 4.2 Phủ mã nguồn (Code Coverage)
3. 4.3 Theo dõi thực thi chương trình

Kiểm thử bảo mật khác Kiểm thử chức năng ở đâu?

	Testing chức năng	**Security Testing**
Mục tiêu	Xác nhận đúng theo đặc tả	Tìm hành vi *ngoài* đặc tả có thể khai thác
Tư duy input	”Điển hình”, hợp lệ	”Ác ý”, cố tình phá vỡ giả định
Tiêu chí thành công	Output đúng mong đợi	Không crash, không rò rỉ, không bị chiếm quyền

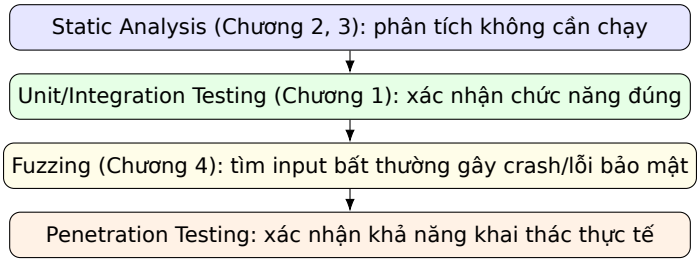
Nhắc lại: Vì sao Testing truyền thống bỏ sót lỗ hổng (Chương 1)

- Test case chức năng thường thiết kế theo equivalence partitioning/boundary value (Chương 1) nhắm vào input *hợp lý*
- Kẻ tấn công tìm kiếm chính xác những input *không hợp lý* mà lập trình viên không lường trước (input cực dài, ký tự đặc biệt, cấu trúc lồng sâu bất thường)
- Chương 4 giới thiệu kỹ thuật **tự động sinh** loại input này ở quy mô lớn – đó chính là **fuzzing**

Các loại Security Testing

- **Fuzzing**: tự động sinh lượng lớn input để tìm input gây crash/lỗi (trọng tâm của chương này)
- **Penetration Testing**: chuyên gia mô phỏng tấn công thực tế, thường thủ công kết hợp công cụ
- **Vulnerability Scanning**: quét tự động theo cơ sở dữ liệu lỗ hổng đã biết (CVE)
- **Red Teaming**: mô phỏng toàn diện một cuộc tấn công thực sự, bao gồm cả yếu tố con người (social engineering)

Vị trí Fuzzing trong quy trình phát triển



Lịch sử ngắn: Fuzzing ra đời như thế nào

- 1988: Giáo sư Barton Miller (Wisconsin) phát hiện nhiều chương trình Unix crash chỉ bằng cách gửi *dữ liệu ngẫu nhiên* qua kết nối modem bị nhiễu – khởi nguồn thuật ngữ ”fuzz”
- Ban đầu chỉ là input hoàn toàn ngẫu nhiên (random fuzzing) – hiệu quả bất ngờ trong việc tìm lỗi crash
- Ngày nay: fuzzing hiện đại (AFL, libFuzzer, honggfuzz) dùng coverage-guided và symbolic execution, hiệu quả hơn hàng nghìn lần so với random fuzzing thuần túy

Case study: OSS-Fuzz (Google)

- Dự án fuzzing liên tục miễn phí cho phần mềm mã nguồn mở quan trọng (OpenSSL, curl, sqlite, FFmpeg...)
- Đã phát hiện **hàng chục nghìn** lỗ hổng/lỗi kể từ khi ra mắt (2016), bao gồm nhiều lỗi bộ nhớ nghiêm trọng đã học ở Chương 2
- Minh chứng thực tế cho hiệu quả của fuzzing quy mô lớn, chạy liên tục 24/7 trên hạ tầng đám mây

Mục tiêu cụ thể của Security Testing/Fuzzing

- Phát hiện **crash** (segmentation fault, assertion failure) – dấu hiệu rõ ràng của lỗi bộ nhớ (Chương 2)
- Phát hiện vi phạm **sanitizer** (ASan, UBSan đã học ở Chương 2) khi chạy với input do fuzzer sinh ra
- Phát hiện **hang** (treo chương trình) – có thể là DoS tiềm ẩn (Chương 1)
- Phát hiện vi phạm **invariant tùy chỉnh** (assertion do lập trình viên tự viết để kiểm tra logic nghiệp vụ)

Test Oracle trong Fuzzing (nhắc lại Chương 1)

- Fuzzing thường **không cần oracle phức tạp** – oracle đơn giản nhất là ”chương trình có crash hay không?”
- Đây là lý do fuzzing giải quyết được một phần *Test Oracle Problem* đã học ở Chương 1: không cần biết ”kết quả đúng” là gì, chỉ cần biết ”có sự cố hay không”
- Với lỗi logic tinh vi hơn (không crash), vẫn cần assertion/invariant cụ thể để fuzzer phát hiện được

Bài tập: Xác định input ”ác ý” cho một hàm

```
void process_name(char *name, int max_len) {
    char buffer[64];
    strncpy(buffer, name, max_len);
}
```

Với tư duy security testing (không phải testing chức năng thông thường), hãy đề xuất 3 input ”ác ý” đáng thử cho hàm này.

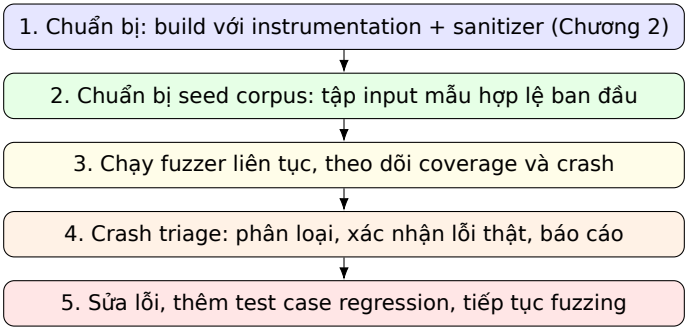
Đáp án: Xác định input ”ác ý”

1. `max_len = 1000` (lớn hơn nhiều `sizeof(buffer)=64`) – kiểm tra buffer overflow
2. `max_len = -1` (số âm, nếu tham số là `int` có thể ép kiểu bất ngờ thành số dương khổng lồ khi truyền vào `strncpy` nhận `size_t`)
3. `name = NULL` – kiểm tra null pointer dereference

Threat Model cho Fuzzing: Ai là đối tượng tấn công giả định?

- Fuzzing mô phỏng góc nhìn của một **kẻ tấn công bên ngoài** không có quyền truy cập mã nguồn/tài liệu nội bộ (black-box) hoặc **có đầy đủ thông tin** (white-box, sẽ học ở mục 4.6)
- Khác với penetration testing (thường có mục tiêu và phạm vi rõ ràng do khách hàng định nghĩa), fuzzing thường chạy *tự động, liên tục*, không cần con người giám sát trực tiếp
- Phù hợp nhất cho phần mềm xử lý *input có cấu trúc phức tạp* từ nguồn không tin cậy: file parser, network protocol, trình duyệt, codec

Vòng đời của một chiến dịch Fuzzing



Seed Corpus – Vai trò của Input Mẫu Ban Đầu

Định nghĩa

Seed Corpus: tập input hợp lệ ban đầu dùng làm điểm khởi đầu cho fuzzer, thay vì bắt đầu từ input hoàn toàn ngẫu nhiên/rỗng.

Ví dụ

Khi fuzzing một thư viện đọc file PNG, seed corpus nên chứa vài file PNG hợp lệ đa dạng (khác kích thước, độ sâu màu, có/không có metadata) – giúp fuzzer ”hiểu” cấu trúc định dạng nhanh hơn nhiều so với bắt đầu từ byte ngẫu nhiên.

Chất lượng Seed Corpus ảnh hưởng thế nào tới hiệu quả

- Seed corpus đa dạng, bao phủ nhiều ”nhánh chức năng” khác nhau giúp fuzzer đạt coverage cao nhanh hơn đáng kể
- Seed corpus kém đa dạng (vd: chỉ 1 file mẫu) khiến fuzzer mất nhiều thời gian ”tự khám phá” cấu trúc định dạng từ đầu
- Kỹ thuật **corpus minimization**: loại bỏ các seed dư thừa (không đóng góp coverage mới) để tăng tốc độ mỗi vòng lặp fuzzing

Crash vs. Hang vs. Sanitizer Violation

Crash	Chương trình dừng đột ngột (segfault, abort) – dấu hiệu rõ ràng nhất
Hang	Chương trình treo (không phản hồi trong thời gian timeout quy định) – có thể là vòng lặp vô hạn hoặc deadlock
Sanitizer Violation	ASan/UBSan (Chương 2) phát hiện lỗi dù chương trình *chưa* crash ngay lập tức – phát hiện sớm hơn crash thực tế

Fuzzing và trách nhiệm đạo đức (Responsible Disclosure)

- Khi fuzzing tìm ra lỗ hổng trong phần mềm của *bên thứ ba* (không phải sản phẩm của mình), cần tuân thủ quy trình **tiết lộ có trách nhiệm** (responsible disclosure)
- Thông thường: báo cáo riêng tư cho nhà phát triển, cho thời gian hợp lý để vá (thường 90 ngày theo chuẩn công nghiệp), sau đó mới công bố công khai
- Không công bố khai thác (exploit) chi tiết trước khi có bản vá – tránh tạo điều kiện cho kẻ tấn công thực sự lợi dụng

Bài tập: Xây dựng Seed Corpus

Bạn cần fuzzing một hàm phân tích cú pháp JSON tự viết bằng C. Đề xuất 4 loại input nên có trong seed corpus ban đầu.

Đáp án: Xây dựng Seed Corpus

1. JSON hợp lệ đơn giản: `{"key": "value"}`
2. JSON lồng nhau sâu: object/array lồng nhiều cấp
3. JSON với các kiểu dữ liệu khác nhau: số, chuỗi, boolean, null, mảng rỗng, object rỗng
4. JSON với ký tự Unicode/escape đặc biệt: `"\u0041"`, dấu ngoặc kép trong chuỗi

So sánh Fuzzing với các kỹ thuật đã học toàn học phần

	Sound?	**Complete?**
Static Analysis (Ch.2,3)	Có thể (over-approx)	Thường không
BMC (Ch.3)	Có (trong phạm vi k)	Chỉ khi đạt CT
Fuzzing (Ch.4)	Không (chỉ đường đã thử)	Không (crash tìm được luôn là thật)

Fuzzing là kỹ thuật **under-approximation** điển hình (nhắc lại mục 3.2) – bù lại, dễ áp dụng và mở rộng tốt hơn nhiều so với BMC/formal proof cho phần mềm lớn thực tế.

Case study: Heartbleed và Fuzzing (nhìn lại Chương 1)

- Heartbleed (CVE-2014-0160, đã học ở Chương 1) là lỗi *out-of-bounds read* trong xử lý gói tin TLS heartbeat
- Về sau, cộng đồng phát hiện rằng fuzzing đơn giản với coverage feedback (nếu áp dụng sớm hơn cho OpenSSL) có khả năng cao đã phát hiện được lỗi này trước khi bị khai thác thực tế
- Đây là động lực chính khiến Google khởi động dự án OSS-Fuzz (2016) – fuzzing liên tục cho các thư viện nền tảng quan trọng

Fuzzing không phải ”viên đạn bạc”

Lưu ý

Fuzzing hiệu quả nhất với lỗi **gây crash rõ ràng** (memory safety, Chương 2). Với lỗi logic tinh vi (business logic flaw, đã học ở Chương 1) không gây crash, fuzzing cần assertion/invariant tùy chỉnh để phát hiện được – không tự động ”hiểu” ý nghĩa nghiệp vụ của chương trình.

Luyện tập trắc nghiệm – Câu 1–3

Câu 1. Ai được xem là người khởi xướng khái niệm ”fuzzing”?
A. Alan Turing B. Barton Miller C. Edsger Dijkstra D. Donald Knuth
Câu 2. Oracle đơn giản nhất mà fuzzing thường dùng là gì?
A. So sánh với kết quả mong đợi chi tiết B. Chương trình có crash hay không C. Thời gian chạy D. Số dòng code
Câu 3. OSS-Fuzz là dự án của tổ chức nào?
A. Microsoft B. Google C. Meta D. Apple

Đáp án Câu 1–3

1. B (Barton Miller, 1988).
2. B (Chương trình có crash hay không – oracle đơn giản nhất).
3. B (Google).

Tổng kết 4.1

- Security testing khác testing chức năng: tìm hành vi ngoài đặc tả, tư duy ”ác ý” thay vì ”điển hình”
- Fuzzing là kỹ thuật cốt lõi của chương này – tự động sinh input quy mô lớn
- Oracle đơn giản (crash/không crash) giúp fuzzing giải quyết một phần Test Oracle Problem
- OSS-Fuzz minh chứng hiệu quả thực tế ở quy mô công nghiệp

Nhắc lại Code Coverage (từ Chương 1)

- Đã học: statement coverage, branch coverage, path coverage, cyclomatic complexity (Chương 1, mục 1.4)
- Chương 4 sẽ dùng lại các khái niệm này nhưng với vai trò **mới**: làm *tín hiệu phản hồi* để hướng dẫn fuzzer sinh input tốt hơn (coverage-guided fuzzing)

Vì sao Coverage quan trọng với Fuzzing?

Định nghĩa

Nguyên lý cốt lõi: input mới làm tăng coverage (khám phá đường thực thi chưa từng chạy qua) có khả năng cao hơn để dẫn tới hành vi mới, bao gồm cả lỗi mới.

Nếu fuzzer chỉ sinh input ngẫu nhiên hoàn toàn (không quan tâm coverage), nó sẽ lãng phí phần lớn thời gian thử lại các đường thực thi đã biết – coverage feedback giúp tập trung nỗ lực vào vùng mã nguồn ”chưa được khám phá”.

Instrumentation – Cách đo Coverage khi chạy

Định nghĩa

Instrumentation: quá trình chèn thêm mã theo dõi vào chương trình (thường tại compile-time) để ghi lại thông tin về đường thực thi khi chạy.

Ví dụ

Compiler chèn một biến đếm tại mỗi basic block: mỗi khi block đó được thực thi, biến đếm tăng lên – đây chính là cơ chế cơ bản để đo coverage.

Instrumentation bằng GCC/Clang

```
gcc -fprofile-arcs -ftest-coverage -o program program.c
./program
gcov program.c
```

- `-fprofile-arcs -ftest-coverage`: bật instrumentation đo coverage
- Sau khi chạy, `gcov` sinh báo cáo chỉ ra dòng nào đã/chưa được thực thi
- Với fuzzing hiện đại, cờ `-fsanitize-coverage=trace-pc-guard` (Clang) hiệu quả hơn nhiều, được dùng bởi libFuzzer/AFL

Edge Coverage – Chi tiết hơn Branch Coverage

Định nghĩa

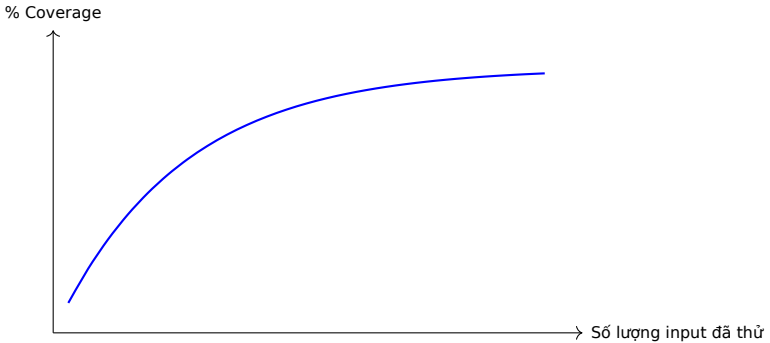
Edge Coverage: theo dõi *từng cạnh cụ thể* trong CFG (Chương 1) đã được đi qua, không chỉ ”nhánh nào” mà còn ”từ block nào tới block nào”.

Edge coverage phân biệt được A→B→C với A→C→B (dù cả 2 đều ”đi qua” 3 block như nhau) – chi tiết hơn branch coverage thông thường, đây là độ đo AFL/libFuzzer thực sự sử dụng.

Coverage Bitmap – Cách AFL lưu trữ Coverage hiệu quả

- AFL dùng một **bitmap** (mảng byte cố định, thường 64KB) để lưu vết các cạnh đã đi qua
- Mỗi cạnh được băm (hash) thành một chỉ số trong bitmap; giá trị tại chỉ số đó tăng lên mỗi khi cạnh được thực thi
- Cách này rất nhanh (O(1) cập nhật) nhưng có thể có *hash collision* (2 cạnh khác nhau ánh xạ cùng vị trí) – đánh đổi tốc độ lấy độ chính xác tuyệt đối

Coverage tăng dần: Ví dụ minh họa



Coverage thường tăng nhanh lúc đầu (dễ tìm đường thực thi mới), sau đó chậm dần (các đường còn lại khó kích hoạt hơn, cần input đặc biệt) – đây là lý do fuzzing dài hạn vẫn có giá trị dù coverage tưởng như đã ”bão hòa”.

Giới hạn của Coverage làm mục tiêu duy nhất

Lưu ý

100% coverage **không đảm bảo** không có lỗi (đã học ở Chương 1) – một dòng có thể chạy qua với giá trị ”an toàn” nhưng vẫn lỗi với giá trị khác. Coverage là *tín hiệu hướng dẫn tốt*, không phải *tiêu chí hoàn thành tuyệt đối*.

Bài tập: Tính Edge Coverage

```
void f(int a, int b) {
    if (a > 0) {
        if (b > 0) { /* A */ }
        else { /* B */ }
    } else {
        if (b > 0) { /* C */ }
        else { /* D */ }
    }
}
```

Với 2 test case: $(a = 1, b = 1)$ và $(a = -1, b = -1)$, có bao nhiêu trong 4 nhánh (A,B,C,D) đã được bao phủ?

Đáp án: Tính Edge Coverage

- $(a = 1, b = 1)$: đi qua nhánh **A**
- $(a = -1, b = -1)$: đi qua nhánh **D**
- Tổng cộng: 2/4 nhánh được bao phủ (50%) – còn thiếu B $(a > 0, b \leq 0)$ và C $(a \leq 0, b > 0)$

Function-level Coverage – Mức thô nhất

Định nghĩa

Function Coverage: đo tỷ lệ các hàm trong chương trình đã được gọi ít nhất một lần.

Đây là độ đo coverage *thô nhất* (ít chi tiết nhất) – một hàm có thể được ”gọi” (tính là covered) nhưng chỉ chạy qua một phần nhỏ logic bên trong nó. Hữu ích để có cái nhìn tổng quan nhanh, nhưng không đủ chi tiết để hướng dẫn fuzzing hiệu quả.

So sánh đầy đủ các mức Coverage

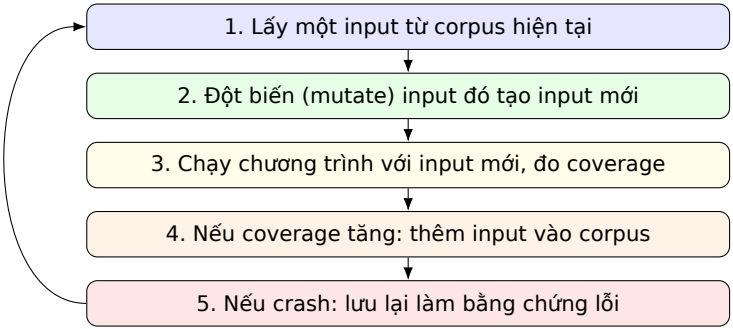
Mức độ	Chi tiết	Dùng cho Fuzzing?
Function	Hàm nào được gọi	Quá thô
Statement/Line	Dòng nào được chạy	Trung bình
Branch	Nhánh nào được chạy	Tốt
Edge	Cạnh cụ thể trong CFG	Rất tốt (AFL dùng)
Path	Toàn bộ đường đi	Quá chi tiết (bùng nổ tổ hợp)

Coverage Report – Đọc hiểu báo cáo thực tế

```
File 'parser.c'
Lines executed:87.50% of 120
Branches executed:72.30% of 65
Taken at least once:68.10% of 65
No branch information for these lines: 3
```

Báo cáo gcov phân biệt rõ ”branch executed” (nhánh đã chạy tới) và ”taken at least once” (thực sự rẽ theo nhánh đó) – chi tiết quan trọng để đánh giá đúng chất lượng bộ test/seed corpus.

Coverage-guided Fuzzing – Vòng lặp chính



Vì sao ”Coverage tăng ⇒ giữ lại input” là chiến lược tốt

- Nguyên lý: nếu input mới khám phá đường thực thi chưa từng thấy, nó có ”tiềm năng” dẫn tới các biến thể input khác cũng mới lạ (qua đột biến tiếp theo)
- Đây là một dạng **thuật toán tiến hóa** (evolutionary algorithm): corpus là ”quần thể”, coverage mới là ”độ thích nghi” (fitness), đột biến là ”sinh sản có biến dị”
- Input không tăng coverage thường bị loại bỏ (không thêm vào corpus) để tránh lãng phí chu kỳ fuzzing vào các biến thể ”vô ích”

Coverage Plateau – Khi Coverage ngừng tăng

- Sau một thời gian, coverage thường *chững lại* (plateau) – các đường thực thi còn lại khó kích hoạt hơn nhiều (cần điều kiện đầu vào phức tạp, cấu trúc lồng sâu)
- Dấu hiệu cần: (1) đổi seed corpus đa dạng hơn, (2) dùng kỹ thuật white-box/symbolic execution (mục 4.6) để ”phá” các điều kiện khó, hoặc (3) fuzzing lâu hơn với nhiều tài nguyên tính toán hơn

Bài tập: Phân tích Coverage Report

Một báo cáo coverage cho thấy hàm `parse_header()` đạt 100% statement coverage nhưng chỉ 60% branch coverage. Điều này gợi ý điều gì về chất lượng bộ test hiện tại?

Đáp án: Phân tích Coverage Report

- 100% statement nhưng chỉ 60% branch nghĩa là: mọi *dòng lệnh* đã được chạy qua, nhưng nhiều *điều kiện rẽ nhánh* chỉ được thử với một giá trị (true hoặc false, không cả hai)
- Gợi ý: cần thêm test case/seed để kích hoạt các nhánh `else` hoặc điều kiện phủ định chưa từng thử – các nhánh xử lý lỗi/edge case thường bị bỏ sót theo cách này (liên hệ Chương 1, mục 1.4)

Corpus Distillation – Tối ưu hóa Corpus theo thời gian

Định nghĩa

Corpus Distillation: định kỳ loại bỏ các input trong corpus không còn đóng góp coverage *độc nhất* (unique) so với các input khác, giữ lại tập nhỏ nhất mà vẫn bảo toàn toàn bộ coverage đã đạt được.

Giúp mỗi vòng lặp fuzzing chạy nhanh hơn (ít input cần thử đột biến hơn) mà không mất đi khả năng khám phá đã tích lũy.

Coverage trong CI/CD: Regression Coverage Tracking

- Ngoài fuzzing, coverage cũng được theo dõi trong CI/CD (đã học ở Chương 1) để đảm bảo test suite không bị ”suy thoái” (giảm coverage) qua thời gian
- Công cụ như Codecov, Coveralls tích hợp vào pull request, cảnh báo nếu thay đổi mã nguồn làm giảm tỷ lệ coverage tổng thể
- Khác biệt với Chương 4: coverage ở đây dùng để *đánh giá chất lượng test suite viết bởi con người*, không phải để *hướng dẫn fuzzer tự động sinh input*

Coverage Targets thực tế trong công nghiệp

- Nhiều tổ chức đặt mục tiêu coverage tối thiểu (vd: 80% line coverage) làm điều kiện *exit criteria* (nhắc lại Chương 1, mục 1.4) trước khi release
- Cảnh báo: đặt mục tiêu coverage cứng nhắc có thể dẫn tới việc viết test ”vô nghĩa” chỉ để tăng số liệu (gọi là ”coverage gaming”) mà không thực sự kiểm tra logic đúng đắn
- Coverage nên được xem là *công cụ hỗ trợ ra quyết định*, không phải mục tiêu tự thân

Luyện tập trắc nghiệm – Câu 4–6

Câu 4. Edge coverage khác Branch coverage ở điểm nào?
A. Edge coverage không cần chạy chương trình B. Edge coverage phân biệt được thứ tự cụ thể giữa các block C. Không có khác biệt D. Edge coverage chỉ dùng cho Java
Câu 5. AFL lưu trữ coverage bằng cấu trúc dữ liệu nào?
A. Cây nhị phân B. Bitmap băm (hash) C. Danh sách liên kết D. Hash table đầy đủ không collision
Câu 6. 100% code coverage có đảm bảo phần mềm không có lỗi không?
A. Có, luôn luôn B. Không, chỉ là tín hiệu hướng dẫn tốt C. Chỉ đúng với C/C++ D. Chỉ đúng nếu dùng CBMC

Đáp án Câu 4–6

4. B (phân biệt thứ tự cụ thể giữa các block, không chỉ ”nhánh nào”).
5. B (Bitmap băm, chấp nhận đánh đổi tốc độ lấy hash collision hiếm gặp).
6. B (Không đảm bảo – chỉ là tín hiệu hướng dẫn tốt, đã học từ Chương 1).

Bài tập tổng hợp: Thiết kế chiến lược Coverage cho dự án thực tế

Một dự án C có 3 module: (1) parser xử lý input mạng, (2) module tính toán nội bộ không nhận input ngoài, (3) module ghi log. Đề xuất mức độ ưu tiên đo coverage cho từng module khi chuẩn bị fuzzing.

Đáp án: Thiết kế chiến lược Coverage

- **Parser (ưu tiên cao nhất)**: attack surface trực tiếp (nhắc lại Chương 2, mục 2.1) – cần edge coverage chi tiết, fuzzing sâu
- **Module tính toán nội bộ (ưu tiên trung bình)**: không nhận input ngoài trực tiếp nhưng vẫn có thể chứa lỗi logic ảnh hưởng an toàn – coverage cơ bản qua unit test đã đủ
- **Module ghi log (ưu tiên thấp)**: rủi ro thấp nhất, có thể chỉ cần function-level coverage

Tổng kết 4.2

- Coverage feedback là động lực chính của fuzzing hiện đại (coverage-guided fuzzing)
- Instrumentation (compile-time) chèn mã theo dõi để đo coverage khi chạy
- Edge coverage chi tiết hơn branch coverage, dùng bitmap để lưu trữ hiệu quả (AFL)
- Coverage cao là tín hiệu tốt nhưng không đảm bảo tuyệt đối không có lỗi

Vì sao cần theo dõi thực thi (Execution Tracing)?

- Ngoài coverage (đã đi qua đâu), fuzzer/công cụ phân tích động cần biết *chi tiết hơn*: giá trị biến, tham số hàm, trạng thái bộ nhớ tại từng thời điểm
- Execution tracing ghi lại lịch sử thực thi chi tiết, phục vụ: gỡ lỗi (Chương 3, mục 3.3), phân tích nguyên nhân crash, và hướng dẫn fuzzer thông minh hơn

Các kỹ thuật theo dõi thực thi

Dynamic Binary Instrumentation	Chèn mã theo dõi vào *mã máy* lúc chạy, không cần mã nguồn (vd: Intel Pin, DynamoRIO)
Compile-time Instrumentation	Chèn mã theo dõi lúc biên dịch (cần mã nguồn, như `-fsanitize-coverage`)
Hardware Tracing	Dùng tính năng phần cứng CPU (Intel PT – Processor Trace) ghi lại luồng thực thi với chi phí runtime cực thấp

Intel Processor Trace (Intel PT) – Giới thiệu

- Tính năng phần cứng trên CPU Intel hiện đại, ghi lại luồng rẽ nhánh (branch) của chương trình với chi phí gần như bằng 0
- Fuzzer hiện đại (kAFL, honggfuzz với chế độ PT) dùng Intel PT để đo coverage cực nhanh, đặc biệt hữu ích khi fuzzing *mã nhị phân đóng gói* (không có mã nguồn để instrument truyền thống)
- Đánh đổi: cần phần cứng hỗ trợ, cấu hình phức tạp hơn compile-time instrumentation

Dynamic Taint Tracking (mở rộng từ Chương 1, 2)

Định nghĩa

Dynamic Taint Tracking: theo dõi *khi chương trình chạy thực tế* xem dữ liệu từ nguồn không tin cậy (tainted) có ảnh hưởng tới các điểm nguy hiểm (sink) hay không – phiên bản *động* của taint analysis tĩnh đã học ở Chương 1, mục 1.4 và Chương 2, mục 2.5.

Ưu điểm so với static taint analysis: chính xác hơn (theo dõi giá trị thực tế, không cần xấp xỉ alias), nhưng chỉ bao phủ đường thực thi *đã chạy qua* (không đầy đủ, tương tự mọi kỹ thuật dynamic).

Ví dụ công cụ: strace – Theo dõi System Call

```
strace -o trace.log ./program input.txt
```

`strace` ghi lại mọi system call (open, read, write, malloc qua brk/mmap...) chương trình gọi tới hệ điều hành – hữu ích để hiểu chương trình tương tác với môi trường bên ngoài thế nào, đặc biệt khi điều tra hành vi đáng ngờ hoặc debug (liên hệ Chương 3, mục 3.3).

Record and Replay – Ghi lại và Phát lại thực thi

- Công cụ như `rr` (Mozilla) ghi lại **toàn bộ** thực thi của chương trình (bao gồm cả nguồn ngẫu nhiên, thời gian hệ thống, kết quả syscall) để có thể *phát lại chính xác* sau này
- Kết hợp với reverse debugging (đã học ở Chương 3, mục 3.3) để điều tra lỗi khó tái hiện (heisenbug)
- Đặc biệt hữu ích khi fuzzer tìm thấy crash: ghi lại toàn bộ trace giúp lập trình viên tái hiện chính xác 100% điều kiện gây lỗi

Crash Triage – Phân loại Crash tự động

- Fuzzer chạy liên tục có thể sinh ra **hàng nghìn** crash – cần tự động phân loại xem crash nào là *duy nhất* (unique) và crash nào chỉ là biến thể của lỗi đã biết
- Kỹ thuật phổ biến: nhóm crash theo *stack trace* (chuỗi lời gọi hàm dẫn tới crash) – các crash có cùng stack trace thường là cùng một lỗi gốc
- Đây là bước xử lý hậu kỳ quan trọng để làm cho fuzzing thực sự hữu ích trong quy trình phát triển thực tế

Bài tập: Thiết kế chiến lược Tracing

Một đội ngũ phát triển muốn fuzzing một thư viện phân tích file ảnh nhị phân đóng gói sẵn (không có mã nguồn). Đề xuất kỹ thuật tracing/coverage phù hợp.

Đáp án: Thiết kế chiến lược Tracing

- Vì không có mã nguồn, **không thể** dùng compile-time instrumentation
- Lựa chọn phù hợp: **Dynamic Binary Instrumentation** (Intel Pin, DynamoRIO) hoặc **Intel PT** (nếu phần cứng hỗ trợ) để đo coverage trên mã nhị phân trực tiếp
- Đây chính là cách các công cụ fuzzing nhị phân thực tế (vd: WinAFL cho Windows binary) hoạt động

Sampling Profiler vs. Instrumentation-based Tracing

Sampling	Định kỳ ”chụp ảnh” trạng thái chương trình (vd: mỗi 1ms) – chi phí thấp, nhưng có thể bỏ lỡ sự kiện ngắn	Với security testing, thường ưu tiên instrumentation (không được bỏ sót lỗi hiếm) hơn sampling (thường dùng cho profiling hiệu năng).
Instrumentation	Ghi lại *mọi* sự kiện quan tâm (mỗi lần gọi hàm, mỗi lần rẽ nhánh) – chi tiết đầy đủ, chi phí cao hơn	

Call Graph Tracing – Theo dõi Chuỗi Lời Gọi Hàm

Định nghĩa

Call Graph: đồ thị biểu diễn quan hệ ”hàm A gọi hàm B” trong toàn bộ chương trình.

Khi kết hợp với execution tracing, call graph động (dynamic call graph) cho biết *chính xác* chuỗi lời gọi dẫn tới một điểm cụ thể trong một lần chạy – khác với call graph tĩnh (chỉ liệt kê khả năng *có thể xảy ra*, không biết *có thực sự xảy ra* hay không).

Memory Access Tracing (nhắc lại liên hệ Chương 2, 3)

- Theo dõi từng lệnh đọc/ghi bộ nhớ (địa chỉ, kích thước, thời điểm) trong khi chương trình chạy
- Đây chính là cơ chế nền tảng của Valgrind/ASan (Chương 2) và ý tưởng shadow memory (Chương 3, mục 3.6) – Chương 4 nhấn mạnh khía cạnh *tracing để hướng dẫn fuzzing* thay vì chỉ phát hiện lỗi đơn thuần
- Có thể dùng để phát hiện *comparison* giữa input và hằng số trong mã nguồn (magic byte, so sánh chuỗi) – gợi ý quan trọng cho fuzzer (mục 4.6)

Ví dụ: Vì sao cần theo dõi Comparison Instructions

```
if (data[0] == 'P' && data[1] == 'N' && data[2] == 'G') {
    parse_png(data);   // Fuzzer rat kho "doan ngau nhien" ra "PNG"!
}
```

Lưu ý

Với input ngẫu nhiên thuần túy, xác suất trúng ngẫu nhiên đúng 3 byte ”PNG” liên tiếp là cực kỳ thấp ($1/256^3$). Fuzzer cần *theo dõi các phép so sánh* này để biết ”cần thử giá trị nào” thay vì đoán mù – đây chính là động lực cho kỹ thuật sẽ học ở mục 4.6 (white-box fuzzing dùng symbolic execution).

Laf-intel và CmpLog – Kỹ thuật thực tế giải quyết vấn đề Magic Byte

- **Laf-intel**: kỹ thuật biến đổi mã nguồn tại compile-time, ”phân rã” các so sánh nhiều byte (vd: so sánh 4 byte một lúc) thành chuỗi so sánh từng byte một – giúp coverage feedback ”dẫn dắt” fuzzer từng bước một thay vì phải đoán đúng cả chuỗi cùng lúc
- **CmpLog** (AFL++): ghi lại các giá trị được so sánh trong quá trình chạy, rồi tự động ”gợi ý” các giá trị này để fuzzer thử trực tiếp
- Cả hai đều là giải pháp *dynamic, heuristic* cho cùng vấn đề mà white-box fuzzing (mục 4.6) giải quyết bằng SMT solving chính xác hơn

Theo dõi Trạng thái Ngăn xếp (Stack Depth Tracing)

- Theo dõi độ sâu ngăn xếp (stack depth) khi chạy giúp phát hiện nguy cơ stack overflow do đệ quy sâu (đã học ở Chương 2)
- Kết hợp với ASan (`-fsanitize=address` đã dùng ở Chương 2) để tự động cảnh báo khi gần đạt giới hạn stack
- Hữu ích khi fuzzing các hàm đệ quy xử lý cấu trúc dữ liệu lồng nhau (JSON, XML) do input người dùng điều khiển độ sâu

Theo dõi thời gian thực thi (Timing/Side-channel, giới thiệu)

- Một số lỗ hổng bảo mật không biểu hiện qua crash mà qua **thời gian thực thi khác biệt** (timing side-channel) – vd: so sánh mật khẩu từng ký tự, dừng sớm khi sai, tiết lộ thông tin qua thời gian phản hồi
- Execution tracing có thể đo thời gian thực thi chi tiết để phát hiện các mẫu hình đáng ngờ này
- Đây là chủ đề chuyên sâu (cryptographic side-channel analysis), vượt ngoài phạm vi cơ bản của học phần nhưng đáng biết là tồn tại

Command Injection Detection qua Tracing (liên hệ Chương 1)

```
system(cmd);   // Neu duoc goi voi cmd chua du lieu "tainted"
```

Dynamic taint tracking (đã giới thiệu) có thể tự động cảnh báo *ngay khi chạy* nếu tham số truyền vào `system()` có nguồn gốc từ input người dùng không qua sanitize – phát hiện command injection (CWE-78, Chương 1) một cách tự động và chính xác hơn static taint analysis thuần túy.

Ftrace và eBPF – Tracing ở mức Hệ điều hành (Linux)

- **ftrace**: framework tracing tích hợp sẵn trong nhân Linux, theo dõi lời gọi hàm kernel, độ trễ, sự kiện lập lịch
- **eBPF** (extended Berkeley Packet Filter): công nghệ hiện đại cho phép chạy chương trình nhỏ, an toàn ngay trong kernel để theo dõi sự kiện hệ thống với chi phí thấp
- Dùng khi cần theo dõi hành vi ở mức *hệ điều hành* (syscall, network, file access) thay vì chỉ trong phạm vi một tiến trình đơn lẻ

Bài tập: Chọn kỹ thuật Tracing phù hợp

Với mỗi tình huống, chọn kỹ thuật tracing phù hợp nhất (compile-time instrumentation, Intel PT, strace, dynamic taint tracking):

1. Cần đo coverage cực nhanh cho một binary đóng gói không có mã nguồn, trên CPU Intel hỗ trợ
2. Nghi ngờ một chương trình gọi lệnh hệ thống nguy hiểm dựa trên input người dùng
3. Đang phát triển mã nguồn C, muốn đo coverage chi tiết nhất cho fuzzing

Đáp án: Chọn kỹ thuật Tracing phù hợp

1. **Intel PT** – chi phí runtime cực thấp, không cần mã nguồn
2. **Dynamic taint tracking** – theo dõi chính xác dữ liệu từ input tới lời gọi `system()`
3. **Compile-time instrumentation** (`-fsanitize-coverage`) – chi tiết nhất, có sẵn mã nguồn để biên dịch lại

Chi phí hiệu năng của các kỹ thuật Tracing

Kỹ thuật	Overhead điển hình	Độ chi tiết
Intel PT	Rất thấp (<5%)	Trung bình (chỉ branch)
Compile-time instrumentation	Thấp-Trung bình (10-30%)	Cao
Dynamic Binary Instrumentation	Cao (2-10x chậm hơn)	Rất cao
Dynamic Taint Tracking	Rất cao (10-50x chậm hơn)	Cực cao

Đánh đổi cố hữu: chi tiết càng cao, chi phí runtime càng lớn – ảnh hưởng trực tiếp tới *tốc độ fuzzing* (số lượng input thử được mỗi giây).

Executions Per Second – Chỉ số quan trọng của Fuzzing

Định nghĩa

Executions per second (exec/s): số lần chương trình mục tiêu được chạy với input mới mỗi giây – chỉ số hiệu năng cốt lõi của một chiến dịch fuzzing.

Kỹ thuật tracing chi tiết hơn (taint tracking) làm giảm exec/s đáng kể – đây là lý do các fuzzer thực tế (AFL, libFuzzer) ưu tiên coverage feedback nhẹ (compile-time instrumentation) làm mặc định, chỉ dùng taint tracking cho các trường hợp đặc biệt khó (magic byte, mục 4.6).

Persistent Mode Fuzzing – Tối ưu hiệu năng

- Thay vì khởi động lại toàn bộ tiến trình cho *mỗi* input thử (tốn kém do overhead khởi tạo hệ điều hành), **persistent mode** chạy hàm mục tiêu lặp lại nhiều lần *trong cùng một tiến trình*
- Tăng exec/s lên hàng chục đến hàng trăm lần so với fork mỗi lần
- Đánh đổi: cần đảm bảo trạng thái toàn cục được reset đúng giữa các lần lặp để tránh ảnh hưởng chéo (state leakage) giữa các lần thử

Bài tập: Phân tích Trade-off hiệu năng

Một đội ngũ đang fuzzing với dynamic taint tracking đầy đủ, đạt 50 exec/s. Chuyển sang compile-time instrumentation cơ bản đạt 5000 exec/s nhưng mất khả năng phát hiện magic byte tự động. Nên chọn chiến lược nào?

Đáp án: Phân tích Trade-off hiệu năng

- Chiến lược tốt nhất thường là **kết hợp cả hai theo giai đoạn**: chạy fuzzing tốc độ cao (5000 exec/s) làm chủ đạo, định kỳ dùng taint tracking/symbolic execution (chậm hơn) chỉ cho các input ”mắc kẹt” (coverage plateau, mục 4.2) để ”phá” các điều kiện khó
- Đây chính là nguyên lý của các fuzzer hiện đại kết hợp black-box và white-box (sẽ học chi tiết ở mục 4.5, 4.6) – không chọn một trong hai tuyệt đối

Tổng kết 4.3

- Execution tracing bổ sung chi tiết ngoài coverage đơn thuần: giá trị biến, syscall, luồng dữ liệu
- Instrumentation có 3 dạng: compile-time, binary (Pin/DynamoRIO), hardware (Intel PT)
- Dynamic taint tracking chính xác hơn static nhưng chỉ bao phủ đường đã chạy
- Record-and-replay (rr) và crash triage là công cụ thực tế quan trọng khi vận hành fuzzing quy mô lớn

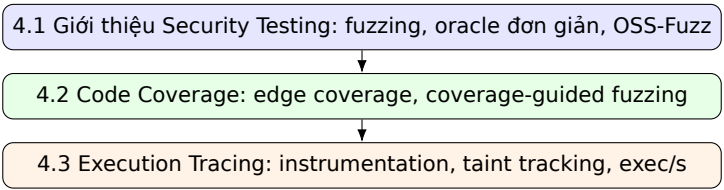
Luyện tập trắc nghiệm – Câu 7–9

Câu 7. Intel PT là gì?
A. Một ngôn ngữ lập trình B. Tính năng phần cứng ghi lại luồng thực thi chi phí thấp C. Một loại sanitizer D. Công cụ quản lý phiên bản
Câu 8. Persistent mode trong fuzzing giúp ích gì?
A. Lưu trữ crash lâu dài B. Tăng exec/s bằng cách tránh khởi động lại tiến trình mỗi lần C. Mã hóa dữ liệu D. Giảm coverage cần thiết
Câu 9. CmpLog (AFL++) giải quyết vấn đề gì?
A. Tăng tốc độ biên dịch B. Giúp fuzzer vượt qua so sánh magic byte khó đoán C. Giảm dung lượng corpus D. Phát hiện memory leak

Đáp án Câu 7–9

7. B (Tính năng phần cứng ghi lại luồng thực thi, chi phí thấp).
8. B (Tăng exec/s bằng tránh khởi động lại tiến trình).
9. B (Giúp vượt qua so sánh magic byte khó đoán ngẫu nhiên).

Bản đồ tri thức Phần 1



Bảng thuật ngữ cốt lõi Phần 1 (1/2)

Fuzzing	Tự động sinh lượng lớn input để tìm crash/lỗi bảo mật
Seed Corpus	Tập input hợp lệ ban đầu làm điểm khởi đầu cho fuzzer
Coverage-guided Fuzzing	Dùng coverage feedback để hướng dẫn sinh input tiếp theo
Edge Coverage	Theo dõi cạnh cụ thể trong CFG, chi tiết hơn branch coverage
Instrumentation	Chèn mã theo dõi vào chương trình để đo coverage/trace

Bảng thuật ngữ cốt lõi Phần 1 (2/2)

Dynamic Taint Tracking	Theo dõi động dữ liệu tainted từ nguồn tới sink khi chạy
Intel PT	Tính năng phần cứng ghi lại luồng thực thi chi phí cực thấp
Crash Triage	Phân loại tự động crash theo stack trace để loại trùng lặp
Exec/s	Số lần chạy chương trình với input mới mỗi giây
Persistent Mode	Chạy lặp lại trong cùng tiến trình để tăng tốc độ fuzzing

Câu hỏi ôn tập tổng hợp Phần 1 (1/2)

1. Vì sao fuzzing giải quyết được một phần Test Oracle Problem đã học ở Chương 1?
2. Coverage-guided fuzzing khác random fuzzing thuần túy ở điểm cốt lõi nào?
3. Vì sao dynamic taint tracking chính xác hơn nhưng chậm hơn nhiều so với compile-time instrumentation?

Câu hỏi ôn tập tổng hợp Phần 1 (2/2)

4. Vấn đề ”magic byte” trong fuzzing là gì và tại sao khó với input ngẫu nhiên?
5. Seed corpus chất lượng tốt ảnh hưởng thế nào tới hiệu quả fuzzing?
6. Vì sao 100% coverage không đảm bảo không có lỗi?

Đáp án gợi ý

1. Vì oracle đơn giản nhất (crash/không crash) không cần biết ”kết quả đúng” là gì, tránh được vấn đề thiếu oracle rõ ràng
2. Coverage-guided ưu tiên giữ lại input làm tăng coverage (khám phá đường mới), tập trung nỗ lực hiệu quả hơn nhiều so với random thuần túy
3. Vì taint tracking phải theo dõi *từng giá trị dữ liệu cụ thể* qua mọi phép toán, trong khi instrumentation chỉ đếm block đã chạy qua – chi tiết hơn luôn đi kèm chi phí cao hơn

Vai trò Phần 1 trong bài tập lớn

Nếu bài tập lớn liên quan tới fuzzing, sinh viên nên: (1) build chương trình mục tiêu với `-fsanitize=address` (Chương 2) và `-fsanitize-coverage=trace-pc-guard`, (2) chuẩn bị seed corpus đa dạng, (3) đo và báo cáo coverage đạt được, (4) crash triage nếu tìm thấy lỗi.

Hỏi đáp

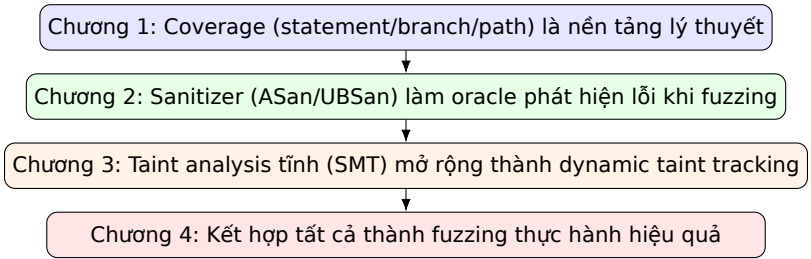
Câu hỏi?

Danh sách kỹ năng thực hành đã đạt được (Phần 1)

Sau Phần 1 của Chương 4, sinh viên đã có thể:

- Giải thích vì sao fuzzing hiệu quả với lỗi bộ nhớ nhưng hạn chế với lỗi logic tinh vi
- Đọc hiểu báo cáo coverage (gcov) và phân biệt statement/branch/edge coverage
- Lựa chọn kỹ thuật tracing phù hợp theo tình huống (có/không có mã nguồn, cần chi tiết cỡ nào)
- Giải thích vấn đề magic byte và các giải pháp thực tế (laf-intel, CmpLog)

Liên hệ tổng thể: Từ Chương 1 đến Chương 4 (Phần 1)



Điểm cần lưu ý khi ôn tập Phần 1

- Phân biệt rõ các mức coverage (function/statement/branch/edge/path) và mối liên hệ với fuzzing
- Hiểu vai trò của coverage *làm tín hiệu phản hồi* chứ không chỉ là chỉ số đánh giá đơn thuần
- Nắm rõ đánh đổi chi phí/độ chi tiết giữa các kỹ thuật tracing khác nhau

Chuẩn bị dự án Python: Xem trước module ch4_dynafuzz

Module `ch4_dynafuzz` (thực hành ở Phần 2) sẽ cài đặt:

- `coverage_tracker.py`: theo dõi coverage cho hàm Python mục tiêu (mô phỏng cơ chế instrumentation)
- `blackbox_fuzzer.py`: fuzzer hộp đen với đột biến ngẫu nhiên có coverage feedback
- `whitebox_fuzzer.py`: fuzzer hộp trắng dùng symbolic execution + Z3 (kết hợp module `ch3_staticsat`)

Câu hỏi thảo luận mở

Một công ty khởi nghiệp có ngân sách hạn chế cho bảo mật. Nếu chỉ được chọn *một* kỹ thuật để áp dụng trong số: static analysis (Chương 2, 3), formal verification (Chương 3), hoặc fuzzing (Chương 4), bạn sẽ tư vấn gì và tại sao?

Gợi ý thảo luận

- Với ngân sách hạn chế, **fuzzing** thường là lựa chọn có tỷ lệ chi phí/hiệu quả tốt nhất: dễ tự động hóa, không cần chuyên gia logic hình thức, phát hiện tốt các lỗi bộ nhớ phổ biến nhất (Chương 2)
- Formal verification (Chương 3) đòi hỏi chuyên môn cao, phù hợp hơn cho module lõi thực sự quan trọng khi có đủ nguồn lực
- Lý tưởng nhất vẫn là kết hợp cả ba theo chiến lược phân lớp rủi ro đã học ở Chương 2, mục 2.1

Tài liệu tham khảo thêm cho Phần 1

- ”Fuzzing: Brute Force Vulnerability Discovery” (Sutton, Greene, Amini)
- AFL++ Documentation – kỹ thuật coverage-guided fuzzing hiện đại
- Google OSS-Fuzz blog – case study thực tế về fuzzing quy mô lớn
- Intel Processor Trace Technical documentation

Vai trò Fuzzing trong chuẩn bị Bài tập lớn

Nếu đề tài bài tập lớn cho phép, hãy thử chạy một fuzzer đơn giản (Python `atheris` hoặc `hypothesis`) lên một hàm xử lý input tự viết – kinh nghiệm thực hành trực tiếp sẽ giúp hiểu rõ hơn nhiều so với chỉ đọc lý thuyết.

Tổng kết ngắn gọn Phần 1

- 4.1: Security testing khác functional testing – tư duy ”ác ý”, oracle đơn giản (crash/không crash)
- 4.2: Coverage (đặc biệt edge coverage) là tín hiệu phản hồi cốt lõi cho fuzzing hiện đại
- 4.3: Tracing cung cấp chi tiết bổ sung (taint, timing, syscall) với đánh đổi hiệu năng rõ ràng

Ghi chú thực hành

Từ Phần 2 trở đi, các ví dụ code sẽ chuyển sang dùng Python (`atheris`, `hypothesis`, thư viện `z3-solver` đã cài từ Chương 3) để sinh viên có thể thực hành trực tiếp trong môi trường `softsec-toolkit` mà không cần công cụ C chuyên biệt như AFL/CBMC.

Chuyển tiếp sang Phần 2

Phần 2 của Chương 4 sẽ trình bày:

- 4.4 Tạo ra các đường thực thi của chương trình – kết hợp symbolic execution (Chương 1, 3) với tracing
- 4.5 Kiểm thử hộp đen (Black-box Fuzzing)
- 4.6 Kiểm thử hộp trắng (White-box Fuzzing) – áp dụng trực tiếp SMT solving (Chương 3)